บทที่ 7

การจัดการหน่วยความจำ

การจัดการหน่วยความจำเป็นหนึ่งในหน้าที่สำคัญของระบบปฏิบัติการ หากหน่วยความจำมีปริมาณให้ใช้งานจำนวนมาก ขีดความสามารถในการทำงานก็จะเพิ่มขึ้นเช่นกัน โปรแกรมที่มีความซับซ้อนและมีความสามารถสูงมักจะต้องการพื้นที่หน่วยความจำที่มาก ซึ่งความต้องการหน่วยความจำของโปรแกรมและโครงสร้างข้อมูลที่ต้องทำงานร่วมกับโพรเซสหรือระบบปฏิบัติการยังคงมีอยู่มาก ดังนั้น จึงเป็นหน้าที่หลักของระบบปฏิบัติการที่จะต้องจัดการหน่วยความจำที่มีอยู่ให้เกิดประโยชน์และประสิทธิภาพสูงสุด

หน่วยความจำจึงถือเป็นส่วนที่สำคัญที่สุดหน่วยหนึ่งในระบบคอมพิวเตอร์ ทำหน้าที่เป็นศูนย์กลางในการดำเนินการต่าง ๆ ให้เป็นไปอย่างราบรื่นและมีประสิทธิภาพสูงสุด ภายในหน่วยความจำจะมีการทำงานหลายส่วน เช่น การทำงานของโปรแกรมจำนวนมาก ซึ่งต้องมีการแบ่งพื้นที่การใช้งานและจัดการด้านต่าง ๆ หากเราสามารถประหยัดหรือใช้งานอย่างมีประสิทธิภาพในพื้นที่หน่วยความจำได้แม้เพียงไม่กี่พันไบต์ ก็อาจสร้างความแตกต่างในการทำงานของแต่ละโปรแกรมได้อย่างมากได้

7.1 แนวคิดเบื้องต้นเกี่ยวกับหน่วยความจำ

หน่วยความจำหลักถือเป็นหัวใจสำคัญของการทำงานในระบบคอมพิวเตอร์สมัยใหม่ โดยภายในจะประกอบด้วยแถวข้อมูลขนาดใหญ่ที่จัดเก็บในรูปแบบของเวิร์ด (word) และไบต์ (byte) ซึ่งแต่ละไบต์จะมีที่อยู่ (address) กำกับไว้เพื่อระบุว่าข้อมูลนั้นถูกจัดเก็บอยู่ที่ตำแหน่งใดในหน่วยความจำ

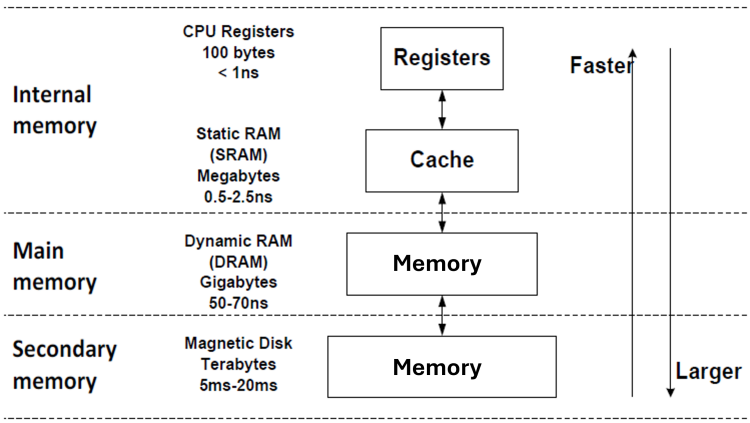
บทบาทสำคัญของหน่วยความจำหลักคือ การใช้เก็บทั้งคำสั่ง (instructions) และข้อมูล (data) ที่จำเป็นต่อการประมวลผล เมื่อโปรแกรมถูกนำมาทำงาน คำสั่งจะถูกส่งเข้าไปยัง หน่วยประมวลผลกลาง เพื่อทำการคำนวณหรือประมวลผล และผลลัพธ์ที่ได้จะถูกส่งกลับมาจัดเก็บไว้ในหน่วยความจำหลักอีกครั้ง เพื่อรอการใช้งานหรือส่งต่อไปยังอุปกรณ์อื่น ๆ

ในมุมมองของนักพัฒนาและโปรแกรมเมอร์ หน่วยความจำที่สมบูรณ์แบบ ควรมีคุณสมบัติ 3 ประการ ได้แก่ 1). ความจุไม่จำกัด (unlimited capacity): สามารถเก็บข้อมูลได้อย่างไม่มีขีดจำกัด, 2). ความเร็วสูง (high speed): เข้าถึงและประมวลผลข้อมูลได้อย่างรวดเร็ว, และ 3). ไม่สูญหาย (non-volatile): ข้อมูลไม่หายไปแม้จะเกิดปัญหาไฟฟ้าดับหรือระบบหยุดทำงาน อย่างไรก็ตาม ในทางปฏิบัติ หน่วยความจำจริงยังไม่สามารถตอบโจทย์ได้ครบทุกด้านพร้อมกัน ทำให้ต้องมีการออกแบบและบริหารจัดการหน่วยความจำให้เหมาะสม เพื่อสร้างสมดุลระหว่าง ความเร็ว ความจุ และความคงทน

ประเภทของหน่วยความจำ

หน่วยความจำถือเป็นหนึ่งในทรัพยากรหลักของระบบคอมพิวเตอร์ ทำหน้าที่สำคัญในการเก็บรักษาทั้งข้อมูล (data) คำสั่ง (instructions) และผลลัพธ์จากการประมวลผล (processing results) เพื่อให้ระบบสามารถทำงานได้อย่างต่อเนื่องและถูกต้อง

โดยทั่วไปแล้ว หน่วยความจำในระบบคอมพิวเตอร์สามารถจำแนกออกเป็น 3 ระดับชั้นหลัก (memory hierarchy) ซึ่งแต่ละระดับมีคุณสมบัติด้าน ความเร็ว (speed) ความจุ (capacity) และ ต้นทุน (cost) ที่แตกต่างกัน ดังแสดงในรูปที่ 7.1



รูปที่ 7.1: ลำดับของหน่วยความจำ

1. หน่วยความจำภายใน (internal memory) หรือแคชเมมโมรี (cache memory): หน่วยความจำประเภทนี้มีความเร็วสูงที่สุดในระบบ ประกอบด้วยรีจีสเตอร์ (registers) และแคช (cache) ซึ่งถูกออกแบบมาเพื่อเก็บคำสั่งหรือข้อมูลที่หน่วยประมวลผลกลางต้องการใช้งานในทันที ข้อดีคือหน่วยประมวลผลกลางสามารถเข้าถึงข้อมูลได้โดยตรงและรวดเร็วมาก จึงช่วยลดเวลาในการประมวลผลลงอย่างมีนัยสำคัญ

2. หน่วยความจำหลัก (main memory) หรือที่เรารู้จักกันในชื่อ RAM (random access memory) ทำหน้าที่เก็บคำสั่งและข้อมูลระหว่างที่โปรแกรมกำลังทำงานอยู่ หน่วยความจำหลักมีความเร็วสูง และ หน่วยประมวลผลกลางสามารถเข้าถึงได้โดยตรงเช่นกัน แต่จะช้ากว่าหน่วยความจำภายในเล็กน้อย จุดสำคัญคือเมื่อปิดเครื่อง ข้อมูลในหน่วยความจำหลักจะหายไป (volatile memory)

3. หน่วยความจำสำรอง (secondary memory): เป็นหน่วยความจำที่มีความเร็วในการเข้าถึงต่ำกว่าสองประเภทแรก แต่มีข้อดีคือสามารถเก็บข้อมูลได้ในปริมาณมาก และเป็นการเก็บข้อมูลแบบถาวร เช่น ฮาร์ดดิสก์ (hard disk), SSD (solid state drive) หรือสื่อเก็บข้อมูลภายนอกอื่น ๆ หน่วยความจำสำรองมักใช้เก็บข้อมูลระยะยาวและโปรแกรมต่าง ๆ ที่รอถูกโหลดเข้าสู่หน่วยความจำหลักเมื่อต้องการใช้งาน

แนวคิดพื้นฐานการจัดการหน่วยความจำหลัก

ระบบปฏิบัติการมีหน้าที่สำคัญในการบริหารจัดการหน่วยความจำหลัก เพื่อให้การทำงานของโปรแกรมต่าง ๆ เป็นไปอย่างมีประสิทธิภาพ โดยสามารถสรุปหน้าที่หลักได้ 4 ประการ ดังนี้

1. ควบคุมและตรวจสอบสถานะของหน่วยความจำ ระบบปฏิบัติการต้องสามารถติดตามได้ว่าตำแหน่งใดในหน่วยความจำถูกใช้งานแล้ว และตำแหน่งใดยังว่างอยู่

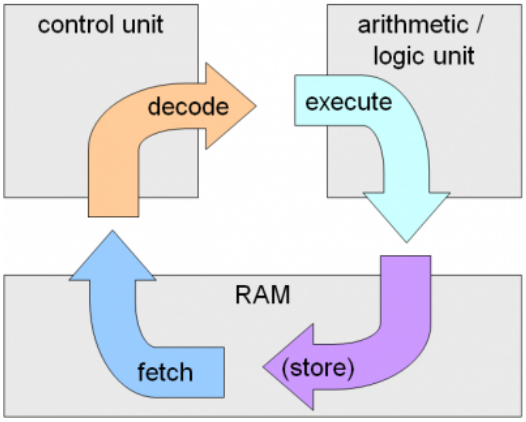
2. ตัดสินใจการจัดสรรหน่วยความจำ เมื่อมีโปรแกรมหรือกระบวนการใหม่เข้ามา ระบบต้องพิจารณาว่าจะจัดสรรพื้นที่ขนาดเท่าใด และตำแหน่งใดในหน่วยความจำที่เหมาะสมที่สุด

3. จัดสรรพื้นที่หน่วยความจำ เมื่อมีการตัดสินใจแล้ว ระบบปฏิบัติการจะจัดสรรพื้นที่หน่วยความจำให้กับโปรแกรมหรือกระบวนการที่ได้รับเลือก

4. คืนพื้นที่หน่วยความจำ เมื่อโปรแกรมหรือกระบวนการสิ้นสุดการทำงาน ระบบจะทำการปลดปล่อยพื้นที่หน่วยความจำที่ถูกใช้งาน เพื่อให้พร้อมสำหรับงานใหม่ในอนาคต

เมื่อโปรแกรมถูกเรียกใช้งาน คำสั่งที่ต้องดำเนินการโดยหน่วยประมวลผลกลางจะต้องถูกดึงขึ้นมาเก็บไว้ในหน่วยความจำหลักเสียก่อน เพื่อให้หน่วยประมวลผลกลางสามารถอ่านและประมวลผลได้อย่างเป็นลำดับ

วงจรการทำงานของคำสั่ง หรือ instruction-execution cycle มีขั้นตอนหลัก ๆ 4 ขั้นตอน (ดังแสดงในรูปที่ 7.2) ดังนี้:



รูปที่ 7.2: การทำงานของรอบคำสั่งเครื่องและการปฏิบัติงานตามคำสั่ง

- ขั้นตอนที่ 1: การดึงคำสั่ง (fetch) หน่วยประมวลผลกลางจะเริ่มต้นด้วยการอ่านคำสั่งจากหน่วยความจำหลัก (main memory) เพื่อเตรียมนำไปประมวลผล

- ขั้นตอนที่ 2: การถอดรหัส (decode) คำสั่งที่ถูกดึงมา จะถูกนำไปถอดรหัสเพื่อแยกความหมายว่าเป็นคำสั่งประเภทใด ต้องใช้ตัวดำเนินการ (operator) หรือข้อมูลใดบ้าง

- ขั้นตอนที่ 3: การประมวลผล (execute) เมื่อถอดรหัสเรียบร้อย หน่วยประมวลผลกลางจะดำเนินการตามคำสั่งนั้น เช่น คำนวณค่าทางคณิตศาสตร์ อ่าน/เขียนข้อมูล หรือควบคุมการทำงานของอุปกรณ์

- ขั้นตอนที่ 4: การจัดเก็บผลลัพธ์ (store) ผลลัพธ์ที่ได้จากการประมวลผลจะถูกบันทึกกลับไปยังหน่วยความจำหลัก หรือรีจีสเตอร์ เพื่อให้สามารถนำไปใช้งานต่อได้

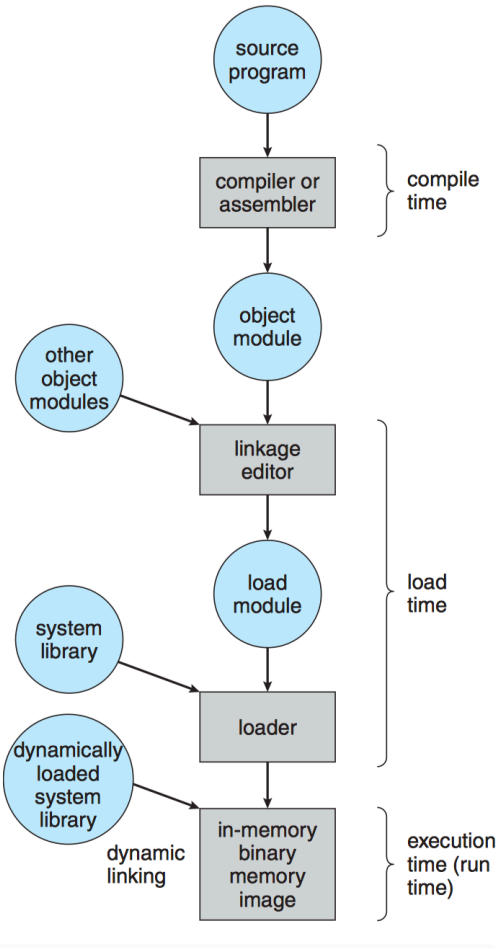
การเชื่อมโยงตำแหน่ง (Address Binding)

ในการทำงานของโปรแกรม คำสั่งและข้อมูลที่ถูกเขียนขึ้นต้องถูกกำหนดให้มีตำแหน่งอยู่ในหน่วยความจำหลักก่อนเสมอ ซึ่งการเชื่อมโยงตำแหน่งนี้สามารถทำได้หลายวิธีและเกิดขึ้นในช่วงเวลาที่แตกต่างกัน

ในระยะแรกของการเขียนโปรแกรม มักใช้ เลขที่อยู่แบบสัมบูรณ์ (absolute address) กล่าวคือ โปรแกรมหรือคอมไพเลอร์จะกำหนดตำแหน่งที่แน่นอนในหน่วยความจำหลัก หากตำแหน่งที่ระบุไม่ว่าง โปรแกรมก็จะไม่สามารถทำงานได้ ทำให้ระบบขาดความยืดหยุ่นและไม่สามารถย้ายตำแหน่ง (relocate) ได้

ต่อมาได้มีการพัฒนาให้ใช้ เลขที่อยู่เชิงสัญลักษณ์ (symbolic address) เช่น ชื่อตัวแปรหรือชื่อซับรูทีน และใช้เลขที่อยู่แบบสัมพัทธ์ (relative address) ในการอ้างอิงตำแหน่งในหน่วยความจำ วิธีนี้ช่วยแก้ปัญหาความไม่ยืดหยุ่น และทำให้โปรแกรมสามารถถูกโหลดไปยังตำแหน่งใด ๆ ในหน่วยความจำหลักได้

กระบวนการเชื่อมโยงคำสั่งและข้อมูล รวมถึงการแปลงเป็นเลขที่อยู่จริงในหน่วยความจำ สามารถเกิดขึ้นได้ในหลายช่วงเวลา ดังนี้ (ดูรูปที่ 7.3):



รูปที่ 7.3: ขั้นตอนต่างๆ ในการเรียกใช้งานของโปรแกรม

1. เวลาแปลโปรแกรม (compile time): หากโปรแกรมระบุเลขที่อยู่แบบสัมบูรณ์ตั้งแต่ต้น เมื่อคอมไพเลอร์ทำการแปล จะได้อ็อบเจ็กต์โปรแกรมที่ผูกกับตำแหน่งจริงในหน่วยความจำหลักทันที จากนั้น absolute loader สามารถโหลดโปรแกรมเข้าสู่หน่วยความจำเพื่อทำงานได้โดยตรง แต่ข้อจำกัดคือ โปรแกรมไม่สามารถย้ายตำแหน่งได้

2. เวลาโหลดโปรแกรม (load time): ในกรณีที่โปรแกรมถูกเขียนโดยอ้างอิงเลขที่อยู่แบบสัมพัทธ์ หลังจากคอมไพเลอร์แปลโปรแกรมแล้ว จะได้อ็อบเจ็กต์โปรแกรมที่ยังไม่ได้ระบุตำแหน่งจริง โหลดเดอร์ (loader) จะทำหน้าที่เลือกพื้นที่ว่างที่เหมาะสมในหน่วยความจำหลัก และแปลงที่อยู่สัมพัทธ์ให้เป็นที่อยู่สัมบูรณ์ โปรแกรมจึงมีความยืดหยุ่น สามารถย้ายตำแหน่งใหม่ได้ทุกครั้งที่ถูกโหลด

3. เวลาดำเนินการ (execution time): บางระบบจะใช้เทคนิค dynamic loading โดยโหลดเฉพาะส่วนของโปรแกรมที่จำเป็นต้องใช้ในขณะนั้นเข้าสู่หน่วยความจำหลัก ส่วนที่เหลือจะเก็บอยู่ในหน่วยความจำสำรอง เมื่อต้องการใช้งานจึงโหลดเพิ่มเติมเข้ามา กระบวนการกำหนดตำแหน่งจริง (binding) จึงเกิดขึ้นแบบไดนามิกระหว่างการทำงานของโปรแกรม ทำให้สามารถใช้หน่วยความจำอย่างมีประสิทธิภาพมากขึ้น

ตัวอย่างที่ 1 : สมมติว่าในโปรแกรมมีตัวแปรชื่อ Addr เมื่อผ่านการคอมไพล์แล้ว คอมไพเลอร์จะกำหนดให้ตัวแปรนี้มีตำแหน่งแบบแอดเดรสย้ายได้ (relocatable address) เช่น อยู่ห่างจากจุดเริ่มต้นของโปรแกรมออกไป 16 ไบต์

หากโปรแกรมถูกโหลดเข้าสู่หน่วยความจำที่ตำแหน่งเริ่มต้น 84,000 โหลดเดอร์จะทำหน้าที่แปลงแอดเดรสย้ายได้ของตัวแปร Addr ให้กลายเป็น แอดเดรสจริง (physical address) ซึ่งในกรณีนี้คือ 84,016

จากตัวอย่างนี้ แอดเดรสของตัวแปรหรือคำสั่งสามารถแบ่งได้เป็น 2 ประเภท คือ:

- absolute address → แอดเดรสจริงทางกายภาพของโปรเซสที่ถูกบรรจุอยู่ในหน่วยความจำ (absolute address เปรียบเสมือน บ้านเลขที่จริง ที่ระบุชัดเจนว่าอยู่ที่ถนนอะไร ซอยไหน หมายเลขเท่าใด ทุกคนสามารถใช้เลขนี้เพื่อหาบ้านได้ตรงตำแหน่งจริง)

- relative address → แอดเดรสที่ได้จากการคอมไพล์ โดยอ้างอิงจากจุดเริ่มต้นของโปรแกรม (relative address เปรียบเสมือน บ้านเลขที่ชั่วคราวในแผนผังจัดสรร ที่เขียนไว้ว่า “บ้านหลังนี้อยู่ห่างจากทางเข้าโครงการ 16 เมตร” เมื่อนำแผนผังไปวางจริงในพื้นที่ ระบบก็จะกำหนดเลขที่บ้านจริงให้ เพื่อใช้งานจริงได้)

การโหลดแบบพลวัต (Dynamic Loading)

การโหลดแบบพลวัต คือ วิธีการที่โปรแกรมเลือกโหลดเฉพาะ ส่วนของโค้ดหรือโมดูลย่อย (routine/module) เข้าสู่หน่วยความจำ เมื่อถึงเวลาที่จำเป็นต้องใช้เท่านั้น แทนที่จะโหลดทุกส่วนตั้งแต่เริ่มต้น โปรแกรมที่ใช้เทคนิคนี้จึงช่วย:

- ประหยัดพื้นที่หน่วยความจำ (memory space)

- เพิ่มประสิทธิภาพการใช้ทรัพยากรของระบบ

ลักษณะสำคัญคือ โปรแกรมย่อยที่ยังไม่ถูกเรียกใช้งานจะไม่ถูกโหลดเข้าสู่หน่วยความจำ ทำให้เหมาะกับโปรแกรมขนาดใหญ่ ที่บางส่วนอาจไม่ถูกใช้งานเลย หรือถูกเรียกใช้งานเพียงบางกรณีเท่านั้น ทั้งนี้ dynamic loading ต้องถูกออกแบบมาจากตัวโปรแกรมเอง ไม่ใช่ความสามารถของระบบปฏิบัติการโดยตรง

ไดนามิกลิงก์และการใช้ไลบรารีร่วมกัน (dynamic linking and shared libraries)

ไดนามิกลิงก์ (dynamic linking) คือ กระบวนการที่โปรแกรมสามารถเชื่อมโยงกับไลบรารี (library) หรือโมดูลภายนอก (external module) ได้ ในช่วงเวลาที่โปรแกรมทำงาน (runtime) ไม่ใช่ในขั้นตอนการคอมไพล์ (compile time) เหมือนการลิงก์แบบปกติ ข้อดีคือ โปรแกรมสามารถเรียกใช้ เวอร์ชันล่าสุดของไลบรารี ได้ทันทีโดยไม่ต้องคอมไพล์ใหม่ทั้งหมด อีกทั้งยังช่วยลดขนาดไฟล์โปรแกรมลง เพราะไม่ต้องบรรจุโค้ดของไลบรารีทั้งหมดเข้าไปในตัวโปรแกรม

การใช้ไลบรารีร่วมกัน (shared libraries) คือ เทคนิคที่อนุญาตให้หลายโปรแกรมสามารถใช้ไลบรารีชุดเดียวกันได้พร้อมกัน โดยแต่ละโปรแกรมสามารถเข้าถึงและเรียกใช้ฟังก์ชันในไลบรารีนี้ได้ โดยไม่จำเป็นต้องก็อปปี้โค้ดใส่ไว้ในไฟล์โปรแกรมของตนเอง ส่งผลให้เกิดการประหยัดทั้ง พื้นที่หน่วยความจำ (memory space) และ เวลาในการโหลด (loading time)

ดังนั้น ไดนามิกลิงก์และการใช้ไลบรารีร่วมกันจึงทำงานควบคู่กัน ดังนี้:

- เมื่อโปรแกรมที่ใช้ dynamic linking เริ่มทำงาน ระบบปฏิบัติการจะทำการเชื่อมโยง (link) โปรแกรมเข้ากับไลบรารีที่จำเป็น ในระหว่างการรัน (runtime) โดยจะโหลด shared library ที่เกี่ยวข้องเข้าสู่หน่วยความจำ
- shared library ที่ถูกโหลดแล้วสามารถถูกใช้งานโดยหลายโปรแกรมพร้อมกัน → ช่วยลดการใช้หน่วยความจำและเพิ่มประสิทธิภาพโดยรวมของระบบ

ไดนามิกลิงก์และการใช้ไลบรารีร่วมกันถูกใช้อย่างแพร่หลายในระบบปฏิบัติการสมัยใหม่ เช่น Windows, Linux และ macOS เนื่องจากช่วยให้:

- การจัดการหน่วยความจำมีประสิทธิภาพมากขึ้น
- การประมวลผลของระบบไม่สิ้นเปลืองเกินความจำเป็น
- การบำรุงรักษาและอัปเดตโปรแกรมทำได้ง่ายและรวดเร็ว (เช่น อัปเดตเวอร์ชันของไลบรารีครั้งเดียว ทุกโปรแกรมที่ใช้จะได้ประโยชน์ทันที)

ตำแหน่งที่ว่างทางกายภาพและทางตรรกะ (Logical Address vs Physical Address)

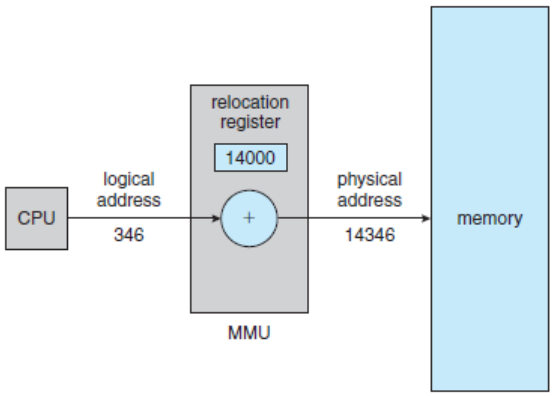
เมื่อโปรแกรมทำงาน หน่วยประมวลผลกลางจะสร้างตำแหน่งอ้างอิงขึ้นมา เรียกว่า ตำแหน่งทางตรรกะ (logical address) ขณะที่ตำแหน่งจริงที่ใช้เก็บข้อมูลในหน่วยความจำจริงจะถูกเรียกว่า ตำแหน่งทางกายภาพ (physical address) โดยทั่วไปในขั้นตอนการคอมไพล์ (compile) และการโหลดโปรแกรม (load) ตำแหน่งทั้งสองจะตรงกัน แต่ในช่วงเวลาที่โปรแกรมทำงานจริง (execution time) มักไม่ตรงกัน ทำให้ตำแหน่งทางตรรกะถูกเรียกว่าตำแหน่งเสมือน (virtual address)

กลุ่มของตำแหน่งทางตรรกะที่โปรแกรมมองเห็นและใช้อ้างอิงจะเรียกว่า พื้นที่เสมือนทางตรรกะ (logical address space) ในขณะที่กลุ่มของตำแหน่งจริงในหน่วยความจำที่สอดคล้องกับตำแหน่งทางตรรกะเหล่านั้นจะเรียกว่า พื้นที่กายภาพ (physical address space) สิ่งนี้ชี้ให้เห็นว่าจริง ๆ แล้วโปรแกรมไม่ได้เข้าถึงหน่วยความจำจริงโดยตรง แต่จะทำงานผ่านตำแหน่งเสมือนที่ถูกแปลงให้สอดคล้องกับหน่วยความจำจริง

เมื่อโปรแกรมถูกโหลดเข้าสู่หน่วยความจำ ตำแหน่งทางตรรกะจำเป็นต้องถูกแปลงเป็นตำแหน่งทางกายภาพ กระบวนการนี้เรียกว่า การย้ายที่อยู่ (relocation) ซึ่งเป็นหน้าที่ของ หน่วยจัดการหน่วยความจำ (memory management unit: MMU) ที่ทำงานอยู่ระหว่างหน่วยประมวลผลกลาง และหน่วยความจำหลัก

หน่วยจัดการหน่วยความจำมีบทบาทสำคัญหลายด้าน ได้แก่ การแปลงตำแหน่งเสมือนให้เป็นตำแหน่งจริง (address translation), การป้องกันไม่ให้โปรแกรมเข้าถึงพื้นที่หน่วยความจำที่ไม่ได้รับอนุญาต (memory protection), การจัดการหน่วยความจำแคชเพื่อเพิ่มความเร็วในการเข้าถึงข้อมูล (cache control) และการควบคุมการสื่อสารบนบัสให้เป็นไปอย่างถูกต้องและปลอดภัย (bus arbitration)

เพื่อให้นักศึกษาเข้าใจได้ง่ายขึ้น อาจเปรียบเทียบตำแหน่งทั้งสองกับ “ที่อยู่หอพัก” โดย logical address เหมือนกับ “แผนผังที่นักศึกษาเขียนว่าอยู่ห้องที่ 5 จากประตู” ส่วน physical address เหมือนกับ “เลขห้องจริงในตึก เช่น ห้องหมายเลข 305” และ หน่วยจัดการหน่วยความจำก็ทำหน้าที่เป็น “ผู้ดูแลหอพักที่คอยจับคู่ระหว่างแผนผังห้องกับเลขห้องจริง” เพื่อให้การเข้าถึงข้อมูลถูกต้องและปลอดภัย



รูปที่ 7.4: การแปลงตำแหน่งในหน่วยความจำ

จากรูปที่ 7.4 จะเห็นได้ว่ารีจีสเตอร์ฐานสามารถถูกปรับเปลี่ยนมาใช้เป็น รีจีสเตอร์สำหรับการย้ายตำแหน่ง (relocation register) เมื่อมีการอ้างอิงตำแหน่งในหน่วยความจำทุกครั้ง ค่าอ้างอิง (logical address) ที่ผู้ใช้ระบุจะต้องถูกนำมาบวกกับค่าที่เก็บอยู่ในรีจีสเตอร์ฐานก่อน จึงจะได้ตำแหน่งจริง (physical address) ตัวอย่างเช่น หากรีจีสเตอร์ฐานมีค่า 14,000 และผู้ใช้ระบุว่าต้องการอ่านข้อมูลจากตำแหน่งที่ 346 ระบบจะทำการบวกค่า 14,000 + 346 ทำให้ได้ตำแหน่งจริงที่ 14,346

จากแนวคิดนี้ จะเห็นว่าโปรแกรมของผู้ใช้ ไม่สามารถทราบค่าตำแหน่งจริงโดยตรง โปรแกรมจะอ้างอิงตำแหน่งเพียงค่าตำแหน่งตรรกะ (logical address) เช่น การสร้างตัวชี้ไปยังตำแหน่งที่ 346 แล้วทำการคำนวณ เก็บค่า อ่านค่า หรือเปรียบเทียบกับตำแหน่งอื่น ๆ โดยอิงจากค่าตำแหน่งตรรกะนั้น ส่วนฮาร์ดแวร์ของเครื่องจะทำหน้าที่แปลงค่าตำแหน่งตรรกะ ให้เป็นตำแหน่งจริงทุกครั้งที่มีการอ้างอิงหน่วยความจำ

แนวคิดการใช้งานตำแหน่งทางตรรกะ (logical address) แทนที่จะอ้างอิงจากตำแหน่งจริง (physical address) โดยตรง ถือเป็นหัวใจสำคัญของการจัดการหน่วยความจำหลัก (main memory management) เนื่องจากช่วยให้ระบบสามารถจัดสรรและควบคุมหน่วยความจำได้อย่างมีประสิทธิภาพ ยืดหยุ่น และปลอดภัยมากยิ่งขึ้น

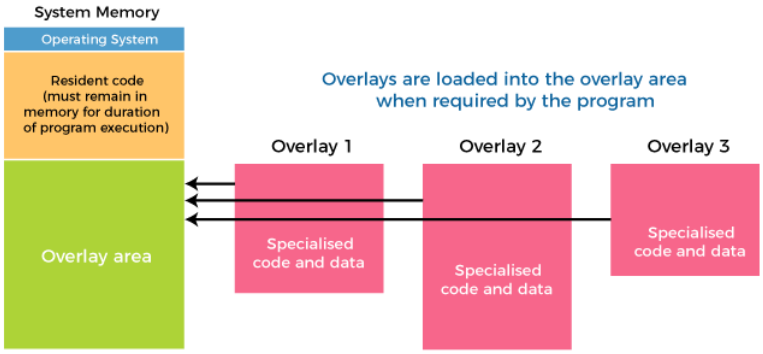
7.2 การจัดการหน่วยความจำหลัก

หน้าที่สำคัญของระบบปฏิบัติการในการจัดการหน่วยความจำหลัก คือการตรวจสอบและควบคุมสถานะของตำแหน่งต่าง ๆ ในหน่วยความจำ ว่าถูกใช้งานหรือยังว่างอยู่ จากนั้นระบบต้องตัดสินใจว่าจะจัดสรรหน่วยความจำขนาดเท่าใดให้กับโพรเซสใด และจะจัดสรรในตำแหน่งใดในหน่วยความจำ เมื่อจัดสรรแล้ว ระบบ

ยังต้องดูแลให้โพรเซสสามารถใช้งานหน่วยความจำได้อย่างถูกต้อง และเมื่อโพรเซสสิ้นสุดการทำงาน หน่วยความจำส่วนนั้นจะต้องถูกปลดปล่อยกลับมาเพื่อให้พร้อมใช้งานกับโพรเซสอื่นต่อไป

วิธีการซ้อนทับ (overlays)

โดยทั่วไป โพรเซสที่จะถูกประมวลผลโดยหน่วยประมวลผลกลาง จะต้องถูกโหลดเข้าสู่หน่วยความจำหลักทั้งหมด และขนาดของโพรเซสต้องไม่ใหญ่กว่าหน่วยความจำหลักที่เหลือจากการหักพื้นที่ที่ใช้โดยระบบปฏิบัติการ อย่างไรก็ตาม หากโพรเซสมีขนาดใหญ่กว่าหน่วยความจำหลัก ก็ยังสามารถทำงานได้ด้วยเทคนิคที่เรียกว่า วิธีซ้อนทับ (overlays) ดังที่แสดงในรูปที่ 7.5



รูปที่ 7.5: วิธีซ้อนทับ

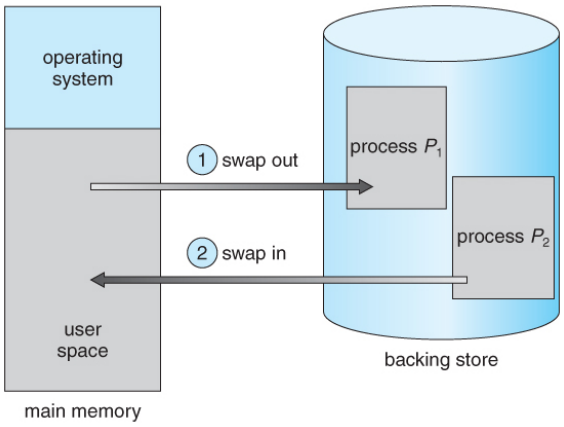
หลักการของการซ้อนทับ คือการแบ่งโพรเซสออกเป็นส่วนย่อยหลายส่วน โดยแต่ละส่วนถูกออกแบบให้สามารถทำงานแยกจากกันได้ และไม่จำเป็นต้องถูกเรียกใช้งานพร้อมกันทั้งหมด ระบบปฏิบัติการจะโหลดเข้าสู่หน่วยความจำเฉพาะส่วนที่จำเป็นต้องใช้งานในขณะนั้น ส่วนที่เหลือของโพรเซสจะถูกเก็บไว้ในหน่วยความจำสำรอง (secondary storage) รอจนกว่าจะถึงเวลาที่ต้องใช้งานจริง จึงจะถูกโหลดเข้ามาแทนที่

วิธีการสับเปลี่ยน (swapping)

เนื่องจากหน่วยความจำหลักมีพื้นที่จำกัด บางครั้งอาจไม่เพียงพอที่จะรองรับการทำงานของทุกโพรเซสได้พร้อมกัน ระบบปฏิบัติการจึงจำเป็นต้องนำโพรเซสบางตัวออกจากหน่วยความจำหลัก เพื่อเพิ่มพื้นที่ว่างสำหรับโหลดโพรเซสอื่นเข้ามาทำงาน กระบวนการนี้เรียกว่า การสับเปลี่ยน (swapping) ดังที่แสดงในรูปที่ 7.6

ตัวอย่างที่ 2: ในระบบที่จัดลำดับการทำงานของโพรเซสตามระดับความสำคัญ โพรเซสที่มีความสำคัญสูงกว่าจะได้รับสิทธิ์ในการทำงานก่อนโพรเซสที่มีความสำคัญต่ำกว่า

ดังนั้น หากโพรเซส P_1 กำลังทำงานอยู่ในหน่วยความจำหลัก แต่มีโพรเซส P_2 ที่มีลำดับความสำคัญสูงกว่าต้องการทำงาน ระบบปฏิบัติการจะต้องนำโพรเซส P_1 ออกจากหน่วยความจำหลักและเก็บไว้ในหน่วยความจำสำรอง ซึ่งกระบวนการนี้เรียกว่า การสับเปลี่ยนออก (swap out) จากนั้นระบบจะโหลดโพรเซส P_2 จากหน่วยความจำสำรองเข้าสู่หน่วยความจำหลัก ซึ่งเรียกว่า การสับเปลี่ยนเข้า (swap in)

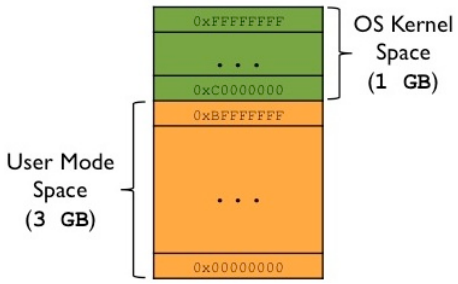


รูปที่ 7.6: วิธีการสับเปลี่ยนการทำงานของโพรเซส

การจัดสรรหน่วยความจำแบบต่อเนื่อง (contiguous memory allocation)

ทุกโปรแกรมจำเป็นต้องใช้หน่วยความจำของระบบในการทำงาน โดยปริมาณการใช้งานจะขึ้นอยู่กับขนาดและลักษณะของโปรแกรม โปรแกรมจะสามารถทำงานได้ก็ต่อเมื่อถูกโหลดเข้าสู่หน่วยความจำ ซึ่งกระบวนการจัดสรรหน่วยความจำ (memory allocation) เป็นหน้าที่หลักของระบบปฏิบัติการในการจัดการให้เกิดประสิทธิภาพสูงสุด

หน่วยความจำหลักต้องรองรับทั้งระบบปฏิบัติการและโพรเซสของผู้ใช้ ดังนั้น โพรเซสแต่ละตัวจะต้องถูกจัดสรรพื้นที่ที่ต่อเนื่องกันในหน่วยความจำหลัก โดยทั่วไป หน่วยความจำหลักจะแบ่งออกเป็นสองส่วน คือ หน่วยความจำระดับต่ำ (low memory) ที่จัดสรรให้กับระบบปฏิบัติการ และหน่วยความจำระดับสูง (high memory) ที่จัดสรรให้กับโพรเซสของผู้ใช้ ดังแสดงในรูปที่ 7.7 รูปนี้แสดงให้เห็นความแตกต่างระหว่างหน่วยความจำของระบบ (kernel mode) และหน่วยความจำที่ใช้ติดต่อกับผู้ใช้งาน (user mode)

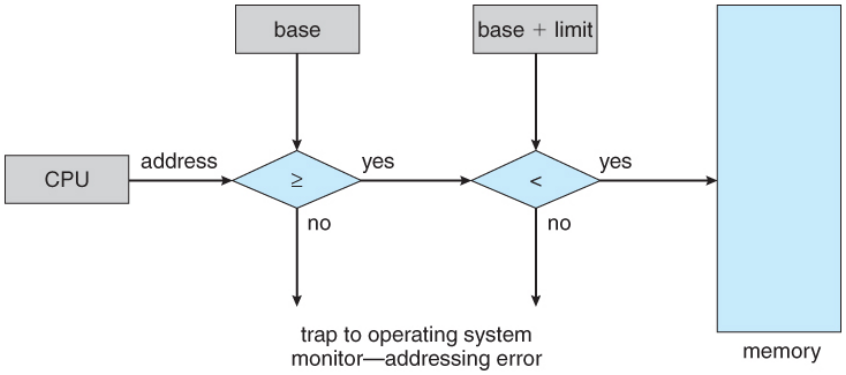


รูปที่ 7.7: วิธีการอ้างอิงตำแหน่งจริงจากหน่วยความจำ

7.2.1 การจัดสรรพื้นที่หน่วยความจำแบบขนาดคงที่ (single-partition allocation)

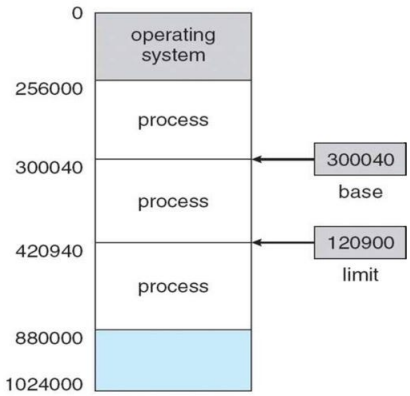
การจัดสรรพื้นที่หน่วยความจำแบบขนาดคงที่ หรือแบบพาร์ติชันเดียว (single-partition allocation) ใช้ระบบรีจิสเตอร์ย้ายตำแหน่ง (relocation-register scheme) เพื่อป้องกันไม่ให้โพรเซสของผู้ใช้รบกวนกันเอง และเพื่อป้องกันไม่ให้โพรเซสของผู้ใช้แก้ไขโค้ดหรือข้อมูลของระบบปฏิบัติการ ระบบนี้ใช้รีจิสเตอร์สองตัว ได้แก่ base register ซึ่งเก็บค่าตำแหน่งเริ่มต้น (physical address) และ limit register ซึ่งเก็บ

ค่าขอบเขตสูงสุดของเลขที่อยู่ตรรกะ โดยทุกการอ้างอิงของโปรแกรมจะต้องมีค่าน้อยกว่าที่กำหนดใน limit register ดังที่แสดงในรูปที่ 7.8



รูปที่ 7.8: ระบบรีจิสเตอร์ย้ายตำแหน่ง

ตัวอย่างเช่น หาก base register ถูกตั้งค่าไว้ที่ตำแหน่ง 300,040 และ limit register กำหนดค่าไว้ที่ 120,900 โปรแกรมจะสามารถเข้าถึงหน่วยความจำได้ตั้งแต่ตำแหน่ง 300,040 จนถึงตำแหน่ง 420,939 ดังที่แสดงในรูปที่ 7.9



รูปที่ 7.9: ตัวอย่างการอ้างอิงของการจัดสรรพื้นที่แบบขนาดคงที่

ในวิธีการนี้ หน่วยความจำจะไม่ถูกแบ่งย่อย แต่ถูกจัดสรรให้เพียงโปรแกรมเดียวในแต่ละครั้ง โดยระบบปฏิบัติการจะอยู่ในพื้นที่หน่วยความจำระดับต่ำ ส่วนโพรเซสของผู้ใช้จะอยู่ในพื้นที่หน่วยความจำระดับสูง การใช้ relocation register และ limit register จะช่วยให้มั่นใจได้ว่าโพรเซสของผู้ใช้จะไม่สามารถเข้ามาแก้ไขพื้นที่ที่เป็นของระบบปฏิบัติการได้ หากมีการอ้างอิงที่เกินขอบเขต ระบบจะทำการสร้างการขัดจังหวะ (interrupt) เพื่อให้ระบบปฏิบัติการเข้ามาจัดการ

ข้อดีของวิธีนี้ คือมีความเรียบง่ายในการจัดการหน่วยความจำ เนื่องจากในแต่ละครั้งจะมีเพียงโปรแกรมเดียวที่ทำงานอยู่ ทำให้ระบบปฏิบัติการมีความซับซ้อนน้อยและง่ายต่อการพัฒนา แต่ข้อเสียคือการใช้หน่วยความจำไม่คุ้มค่า หากโปรแกรมมีขนาดเล็ก พื้นที่ว่างที่เหลือจะไม่สามารถนำไปใช้กับงานอื่นได้ และหากโปรแกรมมีขนาดใหญ่เกินไปจนเกินความจุของหน่วยความจำหลัก โปรแกรมนั้นก็จะไม่สามารถถูกโหลดเข้า

มาทำงานได้ อีกทั้งระบบลักษณะนี้ยังทำงานได้เพียงงานเดียว (single tasking) ส่งผลให้บางช่วงเวลาที่โปรแกรมต้องรออุปกรณ์รับเข้า/ส่งออก หน่วยประมวลผลกลาง (cpu) ก็จะว่างงานโดยไม่ถูกใช้งานอย่างเต็มที่

7.2.2 การจัดสรรพื้นที่หน่วยความจำแบบเป็นพาร์ติชัน (partition allocation)

ข้อจำกัดที่สำคัญของการจัดการหน่วยความจำแบบขนาดคงที่ (single-partition allocation) คือระบบคอมพิวเตอร์สามารถทำงานได้เพียงงานเดียว หรือโพรเซสเดียวในแต่ละครั้ง เพื่อแก้ไขปัญหานี้และเพิ่มประสิทธิภาพของหน่วยประมวลผลกลาง (cpu) ได้มีการพัฒนาระบบคอมพิวเตอร์ให้รองรับการทำงานหลายงานพร้อมกัน ซึ่งเราเรียกว่า การทำงานแบบมัลติโปรแกรมมิ่ง (multiprogramming)

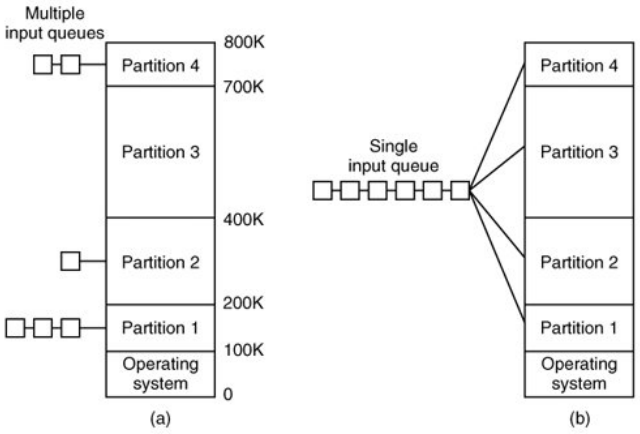
ระบบมัลติโปรแกรมมิ่งช่วยให้หลายโพรเซสสามารถทำงานได้ในเวลาเดียวกัน กล่าวคือ เมื่อโพรเซสหนึ่งอยู่ในสถานะรอการนำเข้า/ส่งออกข้อมูล (i/o) โพรเซสอื่นสามารถเข้ามาใช้ cpu ได้ทันที โดยไม่ต้องเสียเวลารอโหลดโปรแกรมจากหน่วยความจำสำรองเข้าสู่หน่วยความจำหลัก ลักษณะการทำงานเช่นนี้ไม่เพียงแต่ช่วยเพิ่มประสิทธิภาพในการใช้ cpu เท่านั้น แต่ยังเป็นรูปแบบที่ถูกใช้อย่างแพร่หลายในระบบคอมพิวเตอร์สมัยใหม่ ทั้งในเครื่องเซิร์ฟเวอร์และเครื่องลูกข่าย

แนวคิดพื้นฐานของการทำงานหลายโพรเซส คือการแบ่งหน่วยความจำหลักออกเป็นส่วนย่อย ๆ หรือที่เรียกว่า พาร์ติชัน (partition) โดยแต่ละพาร์ติชันจะถูกจัดสรรให้รองรับโพรเซสหนึ่งงาน ดังนั้น จำนวนโพรเซสที่สามารถทำงานพร้อมกันได้จะขึ้นอยู่กับจำนวนพาร์ติชันที่ถูกแบ่งไว้

การกำหนดขนาดพาร์ติชันคงที่ (static partition)

ในวิธีนี้ หน่วยความจำหลักจะถูกแบ่งออกเป็นหลายส่วนที่มีขนาดคงที่ตั้งแต่ก่อนเริ่มการทำงาน โดยการแบ่งส่วนนั้นอาจถูกกำหนดโดยโอเปอเรเตอร์ของระบบคอมพิวเตอร์ หรือโดยระบบปฏิบัติการเอง เมื่อมีโพรเซสใหม่เข้าสู่ระบบ ระบบปฏิบัติการจะตรวจสอบขนาดของโพรเซส และเลือกพาร์ติชันที่เหมาะสมสำหรับการจัดเก็บ (ดังแสดงในรูปที่ 7.10)

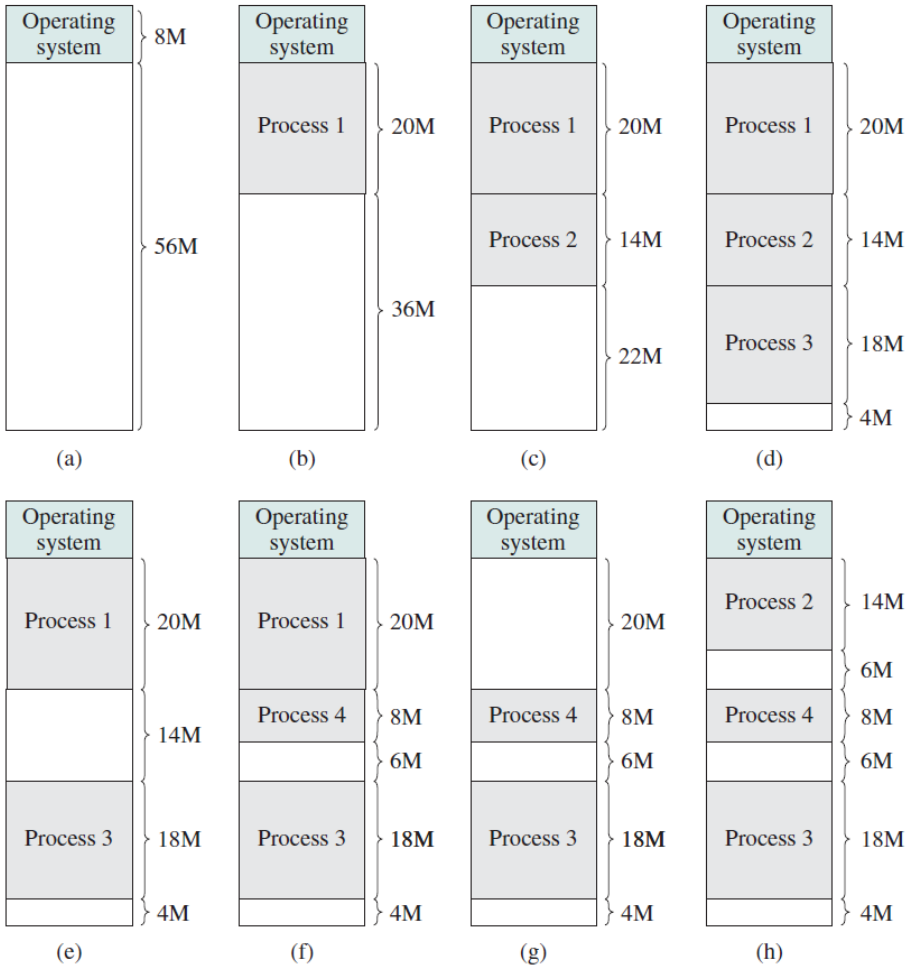
อย่างไรก็ตาม ปัญหาที่อาจเกิดขึ้นคือ ขนาดของโพรเซสไม่พอดีกับขนาดของพาร์ติชัน หากโพรเซสมีขนาดเล็กกว่าพาร์ติชัน จะทำให้มีพื้นที่ว่างบางส่วนที่ไม่สามารถใช้งานได้ เกิดเป็นการสูญเสียพื้นที่หน่วยความจำ หรือที่เรียกว่า internal fragmentation ดังนั้น วิธีนี้จึงเหมาะสำหรับระบบที่สามารถทราบขนาดโพรเซสล่วงหน้า เพื่อให้สามารถกำหนดขนาดของแต่ละพาร์ติชันได้อย่างเหมาะสม



รูปที่ 7.10: การกำหนดขนาดพาร์ติชั่นคงที่: (a) แบบ Multiple in queues (b) Single input queue

การกำหนดขนาดของพาร์ติชันให้เปลี่ยนแปลงได้ (dynamic partition)

การกำหนดขนาดของพาร์ติชันแบบไดนามิก (dynamic) เป็นวิธีการจัดสรรหน่วยความจำหลักที่ยืดหยุ่นมากขึ้น โดยระบบปฏิบัติการจะจัดสรรหน่วยความจำตามขนาดจริงที่โพรเซสต้องการ หากโพรเซสร้องขอพื้นที่ ระบบจะตรวจสอบขนาดของโพรเซสนั้นแล้วจัดสรรพื้นที่ให้พอดี ซึ่งอาจใช้พาร์ติชันที่มีอยู่แล้วหรือแบ่งพาร์ติชันใหม่ขึ้นมาเพื่อรองรับ ดังแสดงในรูปที่ 7.11 เมื่อโพรเซสเสร็จสิ้นการทำงาน พื้นที่ทั้งหมดที่ถูกใช้จะถูกคืนกลับไปยังระบบปฏิบัติการ



รูปที่ 7.11: การกำหนดขนาดของพาร์ติชันให้เปลี่ยนแปลงได้

ตัวอย่างที่ 3: สมมติว่าระบบมีหน่วยความจำหลักทั้งหมด 64 MB ตามรูปที่ 7.11 โดยในรูปที่ 7.11 (a) ส่วนหนึ่งของหน่วยความจำถูกใช้สำหรับระบบปฏิบัติการ และพื้นที่ที่เหลือยังว่างอยู่ เมื่อมีโพรเซสสามงานถูกโหลดเข้ามาใช้งาน ระบบจะจัดสรรพื้นที่จากตำแหน่งถัดจากระบบปฏิบัติการ โดยให้ขนาดพอดีกับความต้องการของแต่ละโพรเซส ดังแสดงในรูปที่ 7.11 (b), (c), และ (d)

ต่อมา เมื่อโพรเซสที่ 4 (มีขนาด 8 MB) ถูกโหลดเข้ามา จะเห็นว่าในรูปที่ 7.11 (d) แม้จะยังมีพื้นที่ว่างแต่ไม่เพียงพอสำหรับโพรเซสที่ 4 ระบบจึงทำการนำโพรเซสที่ 2 ออกจากหน่วยความจำ (swap out) ตามที่แสดงในรูปที่ 7.11 (e) และทำให้โพรเซสที่ 4 สามารถถูกโหลดเข้าไปแทนได้ดังรูปที่ 7.11 (f)

สมมติว่าหลังจากนั้นโพรเซสที่ 2 ต้องการทำงานอีกครั้ง แต่พื้นที่ในหน่วยความจำไม่เพียงพอ ระบบจะ

ต้องดึงโพรเซสที่ 1 ออกมาตามที่แสดงในรูปที่ 7.11 (g) จากนั้นโหลดโพรเซสที่ 2 กลับเข้าไปอีกครั้ง ดังที่แสดงในรูปที่ 7.11 (h)

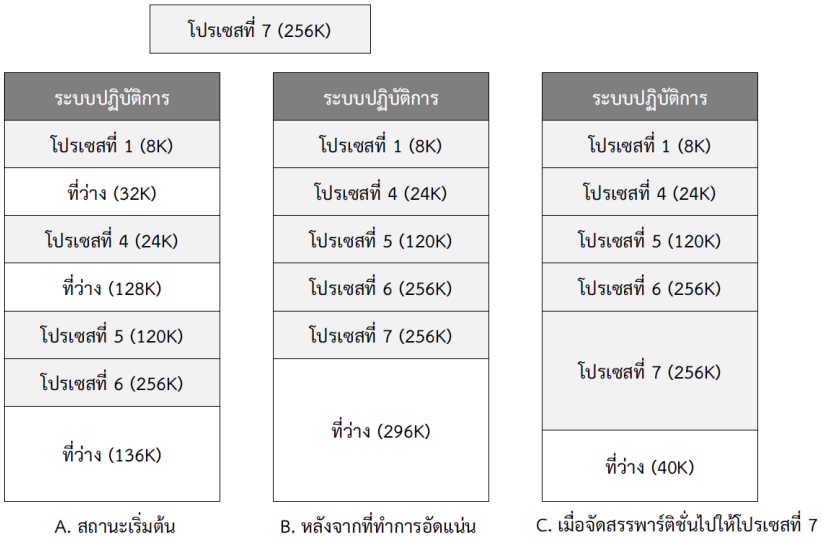
แม้ว่าวิธีนี้จะดูเหมือนทำงานได้อย่างมีประสิทธิภาพ แต่เมื่อระบบทำงานไปนาน ๆ จะเกิด ช่องว่างเล็ก ๆ (external fragmentation) กระจายอยู่ในหน่วยความจำ ช่องว่างเหล่านี้เป็นพื้นที่ที่ไม่สามารถนำไปใช้งานได้จริง แม้ว่าหากรวมพื้นที่ทั้งหมดเข้าด้วยกัน จะมีขนาดใหญ่พอสำหรับบรรจุโพรเซสก็ตาม ปัญหานี้เรียกว่า การกระจัดกระจาย (fragmentation) ซึ่งเป็นข้อเสียสำคัญของการจัดการหน่วยความจำแบบไดนามิก

การจัดสรรพื้นที่หน่วยความจำแบบพาร์ติชันและการย้ายที่อยู่ (partition allocation with relocation)

จากที่ได้อธิบายไปก่อนหน้านี้ การจัดการหน่วยความจำหลักแบบพาร์ติชันมักประสบปัญหาการกระจัดกระจาย (fragmentation) ซึ่งเกิดจากการที่หน่วยความจำถูกจัดสรรและคืนกลับมาหลายครั้ง ทำให้เกิดช่องว่างเล็ก ๆ กระจายอยู่ในหน่วยความจำหลัก ช่องว่างเหล่านี้แม้ว่าจะรวมกันแล้วมีขนาดเพียงพอ แต่ไม่สามารถใช้งานได้เพราะไม่ได้อยู่ติดกัน

เพื่อแก้ปัญหานี้ ระบบสามารถใช้วิธี อัดแน่น (compaction) โดยการรวบรวมพื้นที่ว่างทั้งหมดให้อยู่ติดกัน ทำให้สามารถจัดสรรพื้นที่ขนาดใหญ่พอสำหรับโพรเซสใหม่ได้ อย่างไรก็ตาม การอัดแน่นมีข้อจำกัดสำคัญ คือ เมื่อโพรเซสต่าง ๆ ถูกย้ายตำแหน่งในหน่วยความจำ ค่าอ้างอิงสำคัญ เช่น base address และตำแหน่งของข้อมูลในหน่วยความจำจริง (physical address) จะต้องถูกปรับเปลี่ยนใหม่ ซึ่งกระบวนการนี้เรียกว่า การย้ายที่อยู่ (relocation)

หนึ่งในวิธีแก้ไขคือการนำโปรแกรมผ่านโหลดเดอร์อีกครั้ง เพื่อให้โหลดเดอร์ทำการปรับค่าตำแหน่งใหม่ทั้งหมด แต่ข้อเสียคือแม้บางส่วนของโปรแกรมจะไม่ได้รับผลกระทบจากการอัดแน่น ก็ยังต้องถูกย้ายตำแหน่งใหม่ทั้งหมดเช่นกัน ทำให้เสียเวลาและเพิ่มภาระต่อระบบ



รูปที่ 7.12: วิธีการจัดการหน่วยความจำหลักแบบพาร์ติชันและการย้ายที่อยู่

ดังที่แสดงในรูปที่ 7.12 การอัดแน่นช่วยให้สามารถรวมพื้นที่ว่างจนมีขนาดใหญ่พอสำหรับบรรจุโพรเซสที่ 7 ได้ แต่ก็ต้องแลกมาด้วยการย้ายตำแหน่งของโพรเซสและข้อมูลจำนวนมาก ซึ่งอาจไม่คุ้มค่า โดยเฉพาะอย่างยิ่งหากพื้นที่ที่ได้หลังจากอัดแน่นยังไม่เพียงพอสำหรับโพรเซสที่ร้องขอหน่วยความจำ

7.3 ปัญหาของการจัดการหน่วยความจำหลัก

ในการจัดสรรพื้นที่หน่วยความจำ มักเกิดปัญหาที่ทำให้การใช้งานหน่วยความจำไม่เต็มประสิทธิภาพ โดยเฉพาะกรณีที่มีพื้นที่ว่างขนาดเล็กเกินไปจนไม่สามารถบรรจุโพรเซสใหม่ที่รออยู่ได้ ปัญหาที่พบบ่อยในการจัดการหน่วยความจำหลักมีดังต่อไปนี้

7.3.1 ปัญหาการจัดสรรหน่วยความจำที่เปลี่ยนแปลงได้ (dynamic storage allocation problem)

เนื่องจากข้อกำหนดที่ว่าโพรเซสทั้งหมดต้องถูกโหลดเข้าสู่หน่วยความจำหลัก ทำให้บางส่วนของโพรเซสที่ไม่จำเป็นต้องใช้งานก็ยังถูกโหลดเข้าไป ส่งผลให้เกิดการใช้หน่วยความจำอย่างไม่คุ้มค่า ปัญหานี้เรียกว่า dynamic storage allocation problem ซึ่งเกิดจากการที่ระบบต้องพยายามจัดหาพื้นที่ว่างตามขนาดที่โพรเซสต้องการ

เพื่อแก้ปัญหานี้ ระบบปฏิบัติการจะใช้วิธีการจัดวาง (placement) โดยอาศัยช่องว่างหรือโฮล (hole) ที่เกิดจากการคืนพื้นที่ของโพรเซส วิธีที่นิยมใช้มีดังนี้

1. **first-fit:** เลือกช่องว่างแรกที่มีขนาดเพียงพอ วิธีนี้ทำได้ง่ายและใช้เวลาน้อยที่สุด

2. **best-fit:** ตรวจสอบช่องว่างทั้งหมดแล้วเลือกช่องที่มีขนาดใกล้เคียงกับโพรเซสมากที่สุด วิธีนี้ช่วยลดการสูญเสียพื้นที่ แต่ใช้เวลามากกว่า first-fit และมักทำให้เกิดช่องว่างเล็ก ๆ จำนวนมาก

3. **worst-fit:** เลือกช่องว่างที่มีขนาดใหญ่ที่สุดเพื่อลดปัญหาช่องว่างเล็ก ๆ โดยหวังว่าจะเหลือช่องว่างที่มีขนาดใหญ่พอสำหรับโพรเซสในอนาคต

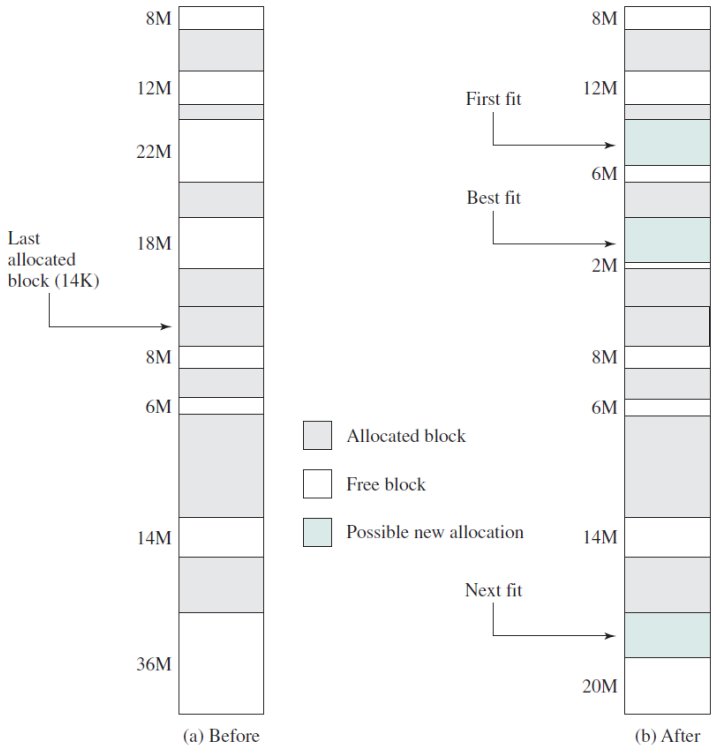
จากการเปรียบเทียบโดยทั่วไป วิธี first-fit และ best-fit มักมีประสิทธิภาพมากกว่า worst-fit ทั้งในด้านความเร็วและการใช้ทรัพยากร ดังแสดงในรูปที่ 7.13

7.3.2 การสูญเสียพื้นที่ย่อยภายนอก (external fragmentation)

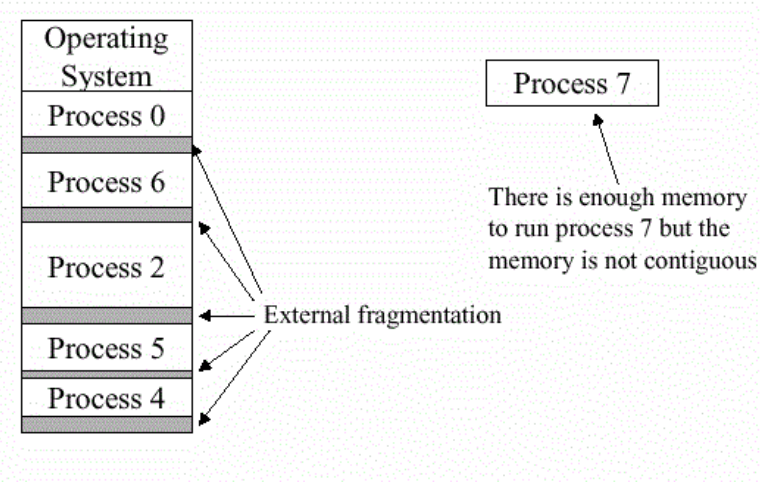
ปัญหานี้เกิดขึ้นเมื่อมีการจัดสรรและคืนพื้นที่หน่วยความจำหลายครั้ง ส่งผลให้พื้นที่ว่างถูกแบ่งออกเป็นส่วนเล็ก ๆ กระจัดกระจาย แม้ว่าพื้นที่ว่างรวมทั้งหมดจะมากพอสำหรับโพรเซสใหม่ แต่เนื่องจากไม่ได้อยู่ติดกัน จึงไม่สามารถจัดสรรได้อย่างมีประสิทธิภาพ

ดังที่แสดงในรูปที่ 7.14 เมื่อมีการคืนพื้นที่และนำมาใช้ใหม่ พื้นที่ว่างจะถูกแบ่งย่อยออกไปเรื่อย ๆ ทำให้เกิดการสูญเสียพื้นที่ย่อยภายนอก ซึ่งส่งผลให้โพรเซสบางงานไม่สามารถทำงานได้ แม้ว่าหน่วยความจำหลักจะยังมีพื้นที่รวมเพียงพออยู่ก็ตาม

เพื่อแก้ไขปัญหานี้ สามารถใช้วิธีการ **อัดแน่น (compaction)** โดยการย้ายตำแหน่งโพรเซสที่ใช้งานอยู่ให้มาติดกัน ทำให้พื้นที่ว่างรวมกันเป็นก้อนเดียว ดังแสดงในรูปที่ 7.15 อย่างไรก็ตาม การอัดแน่นมีต้นทุนสูง ทั้งในแง่ของเวลาและประสิทธิภาพ เนื่องจากต้องย้ายข้อมูลจำนวนมาก



รูปที่ 7.13: เปรียบเทียบการเลือกพื้นที่ด้วยวิธีต่าง ๆ

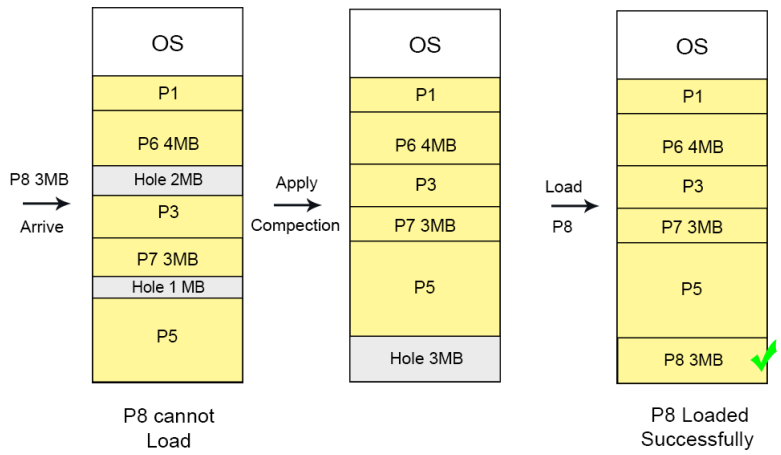


รูปที่ 7.14: พื้นที่ว่างเริ่มแตกเป็นส่วนย่อยเมื่อมีการคืนและจัดสรรใหม่ (fragmentation)

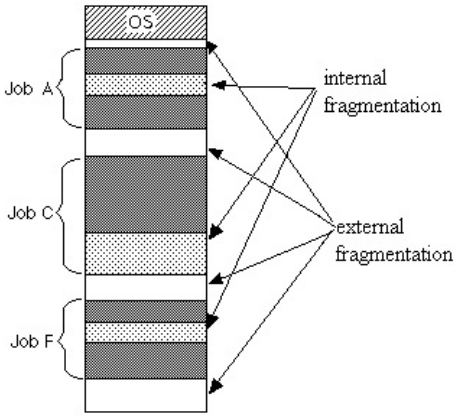
7.3.3 การสูญเสียพื้นที่ย่อยภายใน (internal fragmentation)

ปัญหาการสูญเสียพื้นที่ย่อยภายในเกิดขึ้นเมื่อระบบจองพื้นที่หน่วยความจำให้กับโพรเซสใหญ่กว่าขนาดจริงของข้อมูล ส่งผลให้เกิดช่องว่างที่ไม่ถูกใช้งานภายในพาร์ติชันที่จองไว้ ตัวอย่างเช่น หากโพรเซสมีขนาดเล็กกว่าพื้นที่ที่ถูกจัดสรร จะเกิดการสูญเสียพื้นที่ในส่วนนั้นไปโดยเปล่าประโยชน์

ดังแสดงในรูปที่ 7.16 พื้นที่ที่ถูกจองไว้ไม่ได้ถูกใช้งานทั้งหมด ส่งผลให้เกิดการสูญเสียพื้นที่ย่อยภายใน



รูปที่ 7.15: การอัดแน่นเพื่อลดการสูญเสียพื้นที่ย่อยภายนอก



รูปที่ 7.16: การสูญเสียพื้นที่ย่อยภายใน